

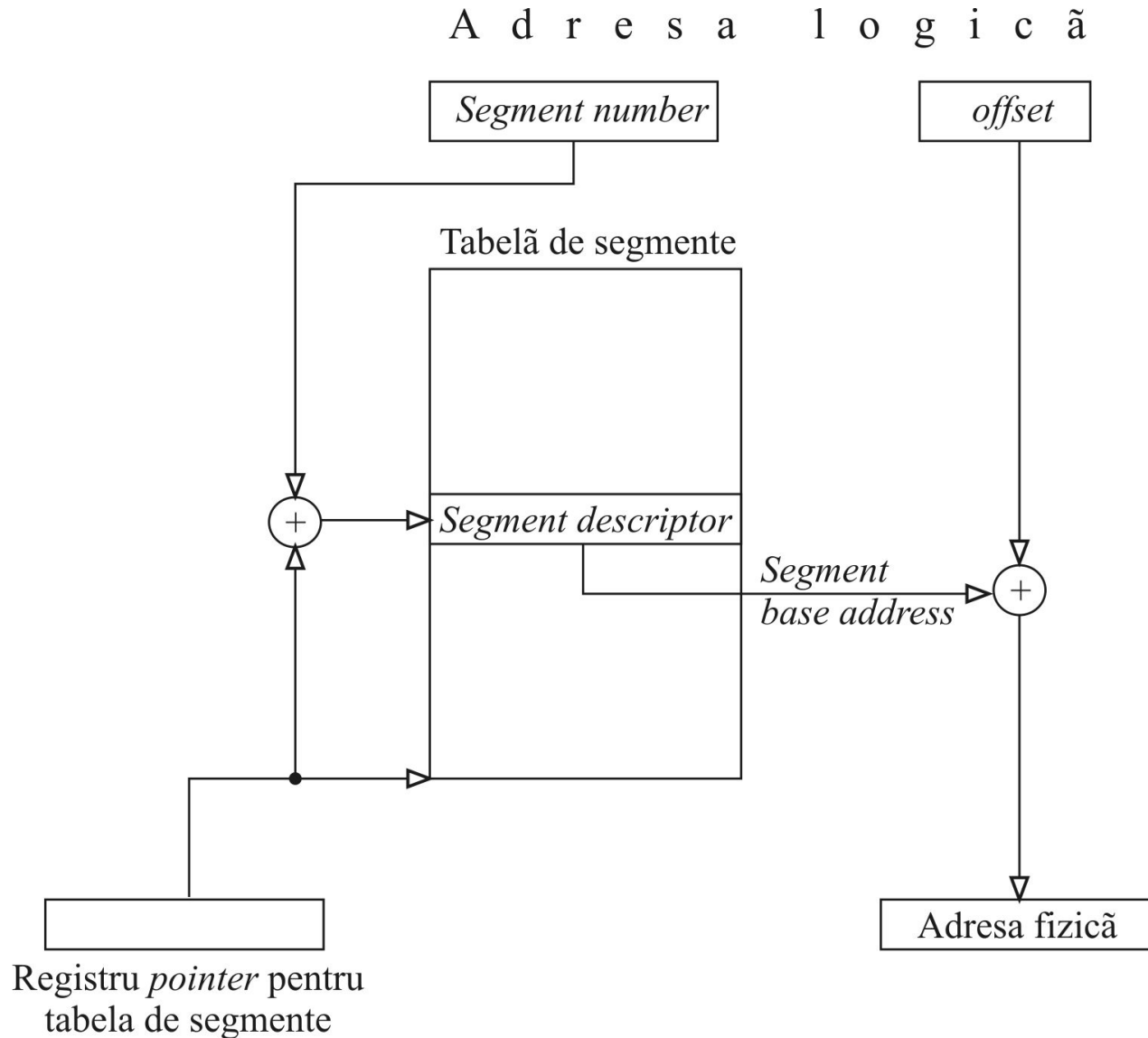
MMU: segmentarea și mecanismul de protecții

1. *Considerații generale*

- Prin segmentare SO poate reloca programele în MP și poate alocă fiecărei aplicații câteva zone de MP, independente și protejate (segment de cod, segmente de date și segment de stivă). Aplicația nu poate accesa decât locații de memorie din propriile segmente. Orice tentativă de acces la o adresă mai mare decât dimensiunea segmentului (acces în afara segmentului), determină procesorul să activeze excepția *segment fault*, iar *handler*-ul excepției (componentă SO) să blocheze (să interzică) accesul respectiv. Segmentarea are deci o motivație *software*.
- Paginarea are o motivație *hardware* și are ca scop implementarea unui sistem automat de gestiune a memoriei ierarhizate MP–MS (sistemul memoriei virtuale).
- Paginile au dimensiuni fixe iar segmentele au dimensiuni variabile deoarece programele au dimensiuni diferite.
- În cazul segmentării, termenul adresă logică este utilizat în locul celui de adresă virtuală și termenul adresă fizică este utilizat în locul celui de adresă reală.
- Adresa de bază a unui segment și *offset*-ul sunt entități separate; programele nu pot modifica baza ci doar *offset*-ul (baza este creată și gestionată de către sistemul de operare și este inaccesibilă aplicațiilor, rulate pe nivel *user*).

MMU: segmentarea și mecanismul de protecții

2. Translatarea adresei prin segmentare



MMU: segmentarea și mecanismul de protecții

3. Descrierea segmentelor și protejarea lor

Descriptorii de segment conțin următoarele informații:

- adresa de bază a segmentului (*segment base address*)
- lungimea segmentului (*segment limit*)
- biți de protecție asociați segmentului respectiv (*control bits*)
- biți pentru algoritmul de înlocuire

Lungimea segmentului:

Segmentele pot avea lungimi (dimensiuni) diferite. Câmpul care definește lungimea segmentului (*segment limit*) are rolul de a preveni accesele în afara segmentului.

Protecția segmentelor

Protecția unui segment din MP înseamnă prevenirea anumitor tipuri de accese la locațiile segmentului respectiv. Tipic, pot fi setate următoarele tipuri de protecții:

- *Read-only*
- *Execute-only*
- *System-only*

MMU: segmentarea și mecanismul de protecții

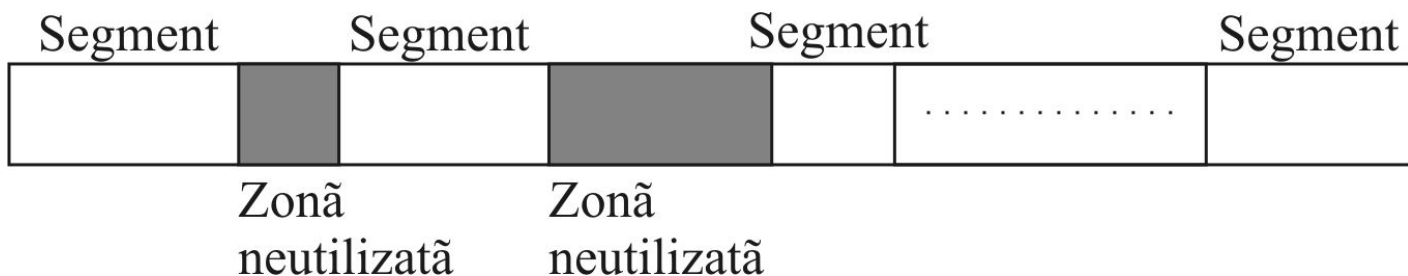
4. Algoritmul de înlocuire și plasare

Algoritmul de înlocuire:

Algoritmul de înlocuire utilizat de către sistemul de segmentare poate fi similar celui implementat în sistemul de paginare exceptând faptul că, în cazul segmentării, algoritmul trebuie să ia în considerare dimensiunea variabilă a segmentelor atunci când alocă spațiu MP pentru noile segmente.

Algoritmul de plasare:

- Dimensiunea variabilă a segmentelor și vehicularea permanentă a acestora între MP și disc conduc la fragmentarea MP
- Algoritmi de plasare (a segmentelor în MP): *first fit*, *best fit* sau *worst fit*.
- Defragmentarea memoriei consumă mult timp de procesare



Fragmentarea memoriei principale produsă de segmentare

MMU: segmentarea și mecanismul de protecții

5. Combinarea mecanismelor de segmentare și paginare

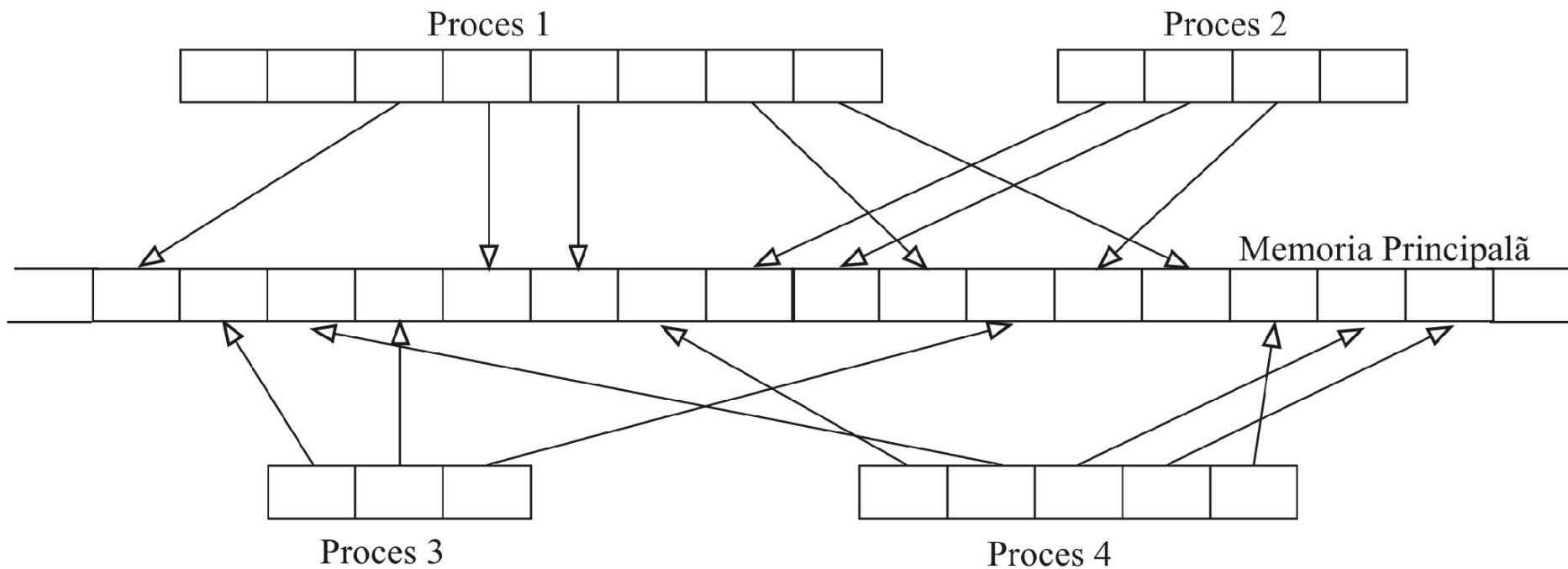
Segmentarea paginată:

Segmentarea și paginarea pot fi combinate, și sunt de regulă combinate, pentru a obține avantajele ambelor mecanisme (structura logică protejată a entităților *software* asigurată prin segmentare și respectiv gestiunea la nivel *hardware* a ierarhiei MP - disc asigurată prin paginare):

- Segmentarea devine segmentare simbolică.
- Fiecare segment este împărțit în pagini de mărimi egale și unitatea de informație care se va transfera între MP și disc va deveni pagina.
- Nu va mai fi necesară încărcarea completă a segmentului în MP; numai paginile într-adevăr necesare (din cadrul acestuia) vor fi transferate în MP.

MMU: segmentarea și mecanismul de protecții

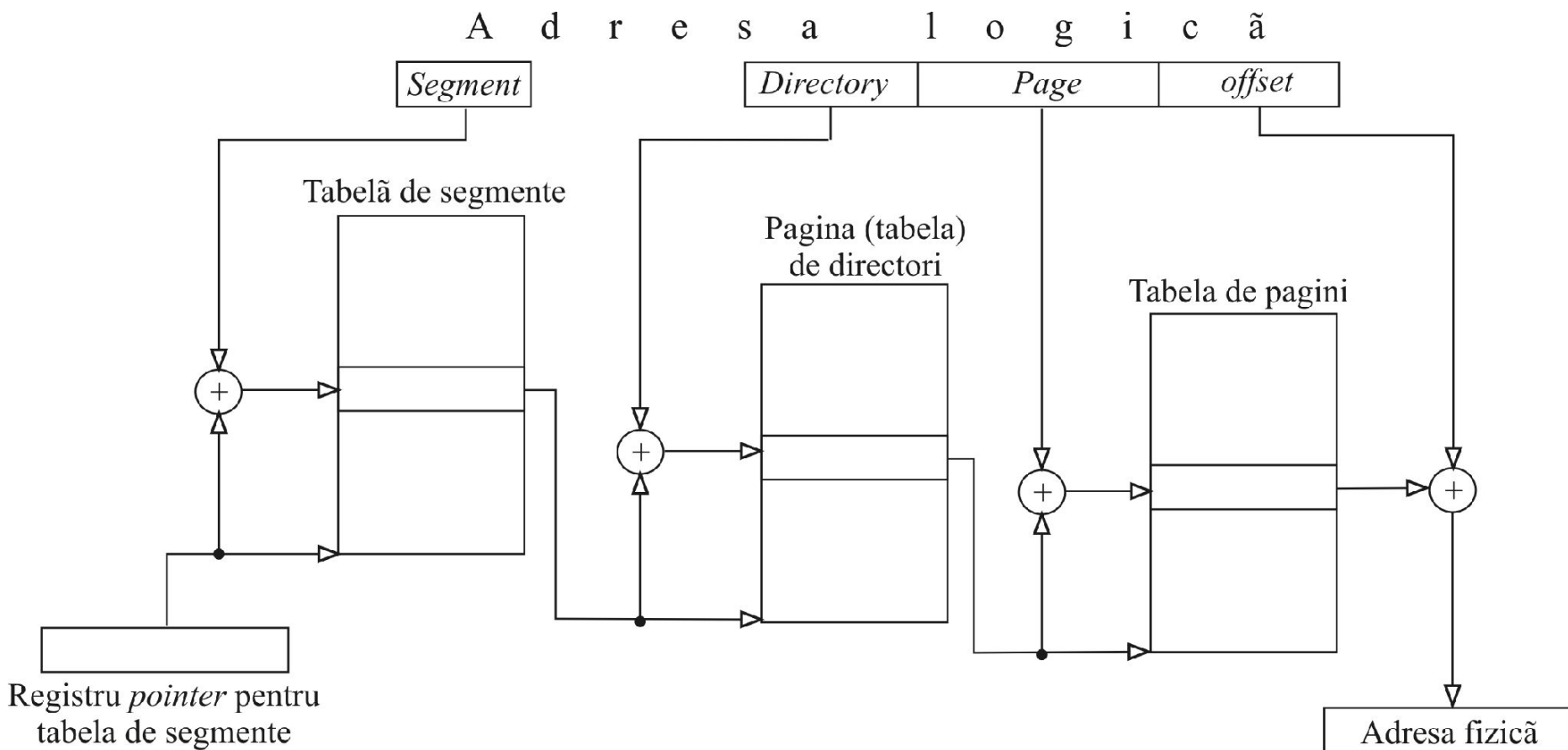
5. Combinarea mecanismelor de segmentare și paginare



Încărcarea segmentelor paginate în memoria principală

MMU: segmentarea și mecanismul de protecții

5. Combinarea mecanismelor de segmentare și paginare



Traducerea adreselor prin segmentare combinată cu paginare pe două nivele

Studiu de caz: Segmentarea și paginarea la procesoarele Intel 80x86

1. Considerații generale

Microprocesorul 8086 (pe 16 biți), lansat de Intel în anul 1978, constituie probabil primul exemplu de microprocesor care încorporează o formă foarte restrictivă de segmentare.

Următoarele microprocesoare din familia 80x86 (80386, 80486, Pentium) au fost dotate cu un MMU extins, dar fiecare extensie a fost proiectată sub restricția compatibilității cu registrele de segment existente la 8086 ceea ce a condus la soluții foarte complicate.

Registrele de segment pe care le regăsim în arhitectura 80x86 sunt:

- CS –registrul segmentului de cod (*code segment register*)
- SS –registrul segmentului de stivă (*stack segment register*)
- DS –registrul segmentului de date (*data segment register*)
- ES –registrul extra segment date (*extra segment register*)
- FS, GS –registre pentru segmente alternative de date

Segmentarea și paginarea la procesoarele Intel 80×86

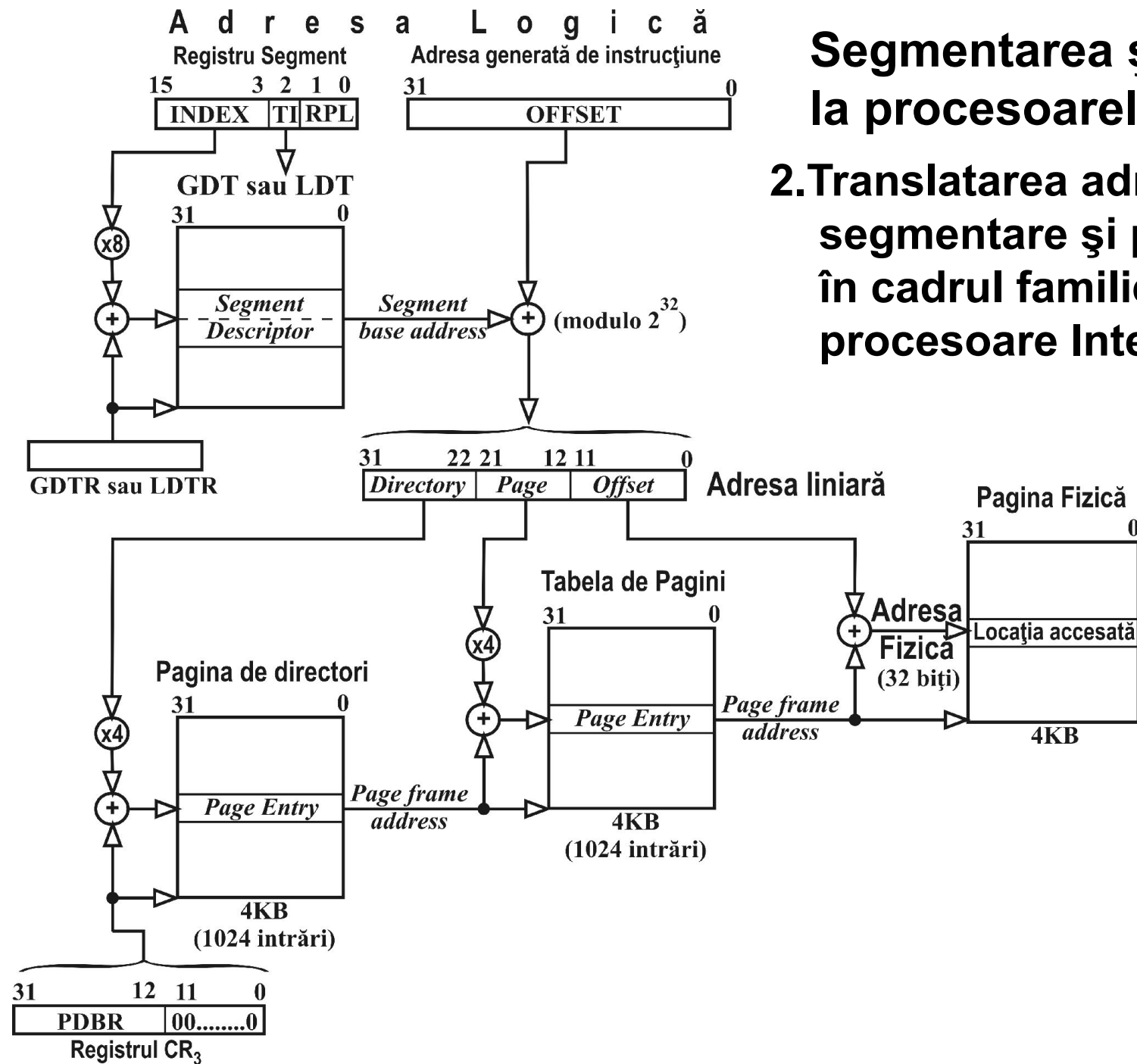
1. Considerații generale

Regulile normale de selecție a segmentelor implementate (în *hardware*) în cadrul arhitecturii Intel 80x86:

Tipul accesului la memorie	Segmentul normal accesat	Registrul de segment utilizat
<i>Fetch</i> instrucțiune	segmentul de cod	CS
a) <i>Fetch</i> operand cu sursa în memorie b) Scriere cu destinația în memorie	segmentul de date	DS
a) Operații PUSH/POP în stivă b) Orice acces la memorie (<i>fetch</i> operand sau <i>write</i>) cu ESP (<i>stack pointer</i>) sau EBP (<i>stack frame base pointer</i>) ca registre de bază	segmentul de stivă	SS
Accese la șiruri destinație în cadrul instrucțiunilor pe șiruri	extra segment de date	ES

Segmentarea și paginarea la procesoarele Intel 80x86

2. Translatarea adresei prin segmentare și paginare în cadrul familiei de procesoare Intel 80x86



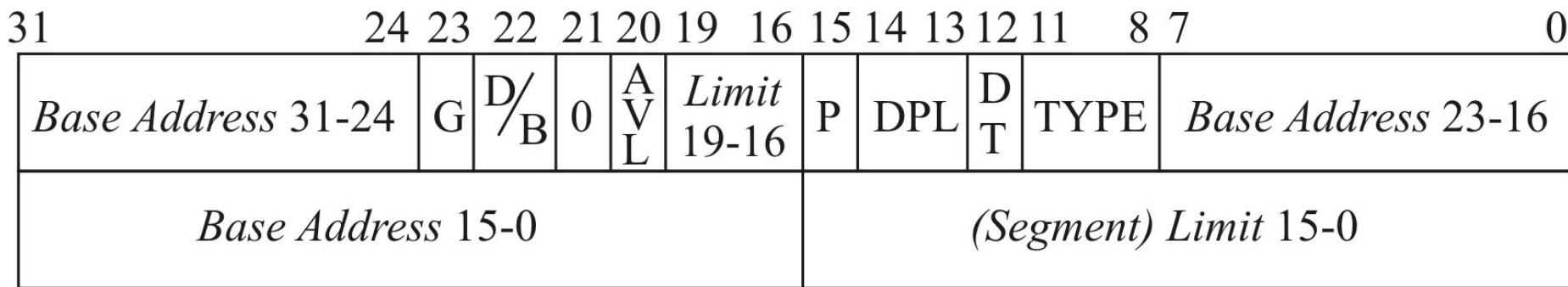
Segmentarea și paginarea la procesoarele Intel 80x86

3. Descriptorii de segment

Descriptorul unui segment (*segment descriptor*) este o structură de date locată în tabelele de traducere aferente sistemului de operare (GDT–*Global Descriptor Table* sau LDT –*Local Descriptor Table*), tabele rezidente în MP.

Descriptorul conține următoarele informații:

- adresa de bază a segmentului (*segment base address*)
- dimensiunea (limita) segmentului (*segment limit*)
- informații de control și stare asociate segmentului respectiv



Formatul descriptorului de segment la procesoarele Intel 80x86

Segmentarea și paginarea la procesoarele Intel 80×86

3. Descriptorii de segment

G – granularity D/B – default/big AVL – available
DPL – descriptor privilege level TYPE – segment type P – present

DT - descriptor type:

- a) DT = 0 pentru segmente sistem (LDT, TSS) pentru care *TYPE* poate lua valorile din tabelul 4.5.
- b) DT = 1 pentru:
 - segmentele de cod (la care *TYPE*=1/C/R/A; vezi tabelul 4.4)
 - segmentele de date (*TYPE*=0/E/W/A; vezi tabelul 4.3)

Semnificațiile biților din câmpul *TYPE* sunt:

C – conforming (segment conformant)
R – read enable (segmentul acceptă accese de tip *fetch* operand)
A – accessed (segment accesat)
E – expand (directia de extindere a segmentului)
W – write enable (segment care acceptă operații de scriere)

Segmentarea și paginarea la procesoarele Intel 80×86

3. Descriptorii de segment

Tipurile de
segmente de date:

TYPE (hexa)	E	W	A	Descriere segment	
0	0	0	0	R-O	R - O = <i>read-only</i>
1	0	0	1	R-O, A	A = <i>accessed</i>
2	0	1	0	R/W	R / W = <i>read/write</i>
3	0	1	1	R/W, A	E - D = <i>expand-down</i>
4	1	0	0	R-O, E-D	
5	1	0	1	R-O, E-D, A	
6	1	1	0	R/W, E-D	
7	1	1	1	R/W, E-D, A	

Tipurile de
segmente de cod:

TYPE (hexa)	C	R	A	Descriere segment	
8	0	0	0	E-O	E-O = <i>execute-only</i>
9	0	0	1	E-O, A	A = <i>accessed</i>
A	0	1	0	E/R	E/R = <i>execute/readable</i>
B	0	1	1	E/R, A	C = <i>conforming</i>
C	1	0	0	E-O, C	
D	1	0	1	E-O, C, A	
E	1	1	0	E/R, C	
F	1	1	1	E/R, C, A	

Segmentarea și paginarea la procesoarele Intel 80×86

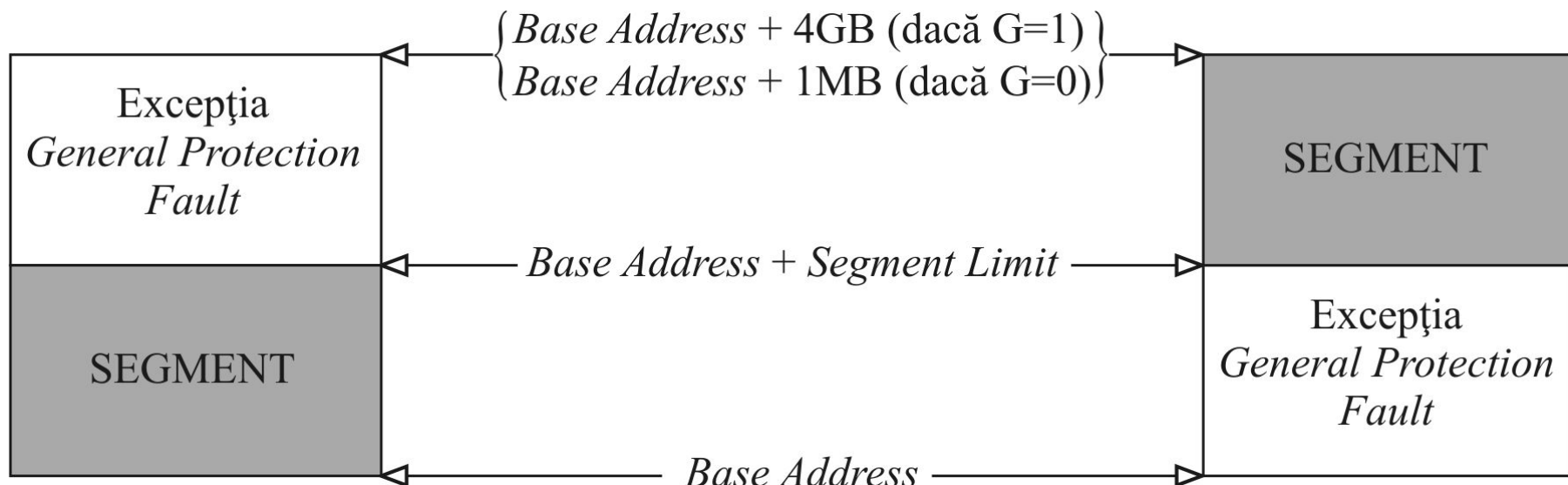
3. Descriptorii de segment

TYPE	Segment (obiect) sistem	
0 0 0 0	Rezervat	
0 0 0 1	80286 TSS	TSS = <i>task state segment</i>
0 0 1 0	LDT	(segmentul de stare <i>task</i>)
0 0 1 1	Busy 80286 TSS	CALL Gate = poartă CALL
0 1 0 0	CALL Gate	Interrupt Gate = poartă de
0 1 0 1	Task Gate	întrerupere
0 1 1 0	80286 Interrupt Gate	Trap Gate = poartă de trap
0 1 1 1	80286 Trap Gate	Task Gate = poartă de task
1 0 0 0	Rezervat	
1 0 0 1	80386 / 486 TSS	
1 0 1 0	Rezervat	
1 0 1 1	Busy 80386 / 486 TSS	
1 1 0 0	80386 / 486 CALL Gate	
1 1 0 1	Rezervat	
1 1 1 0	80386 / 486 Interrupt Gate	
1 1 1 1	80386 / 486 Task Gate	

Semnificațiile câmpului TYPE în cazul segmentelor sistem (segmentelor SO)

Segmentarea și paginarea la procesoarele Intel 80×86

3. Descriptorii de segment



Definirea segmentului prin câmpurile *Base Address*, *Segment Limit*, *G* și *E*.

Segmentarea și paginarea la procesoarele Intel 80x86

4. Intrările în pagini

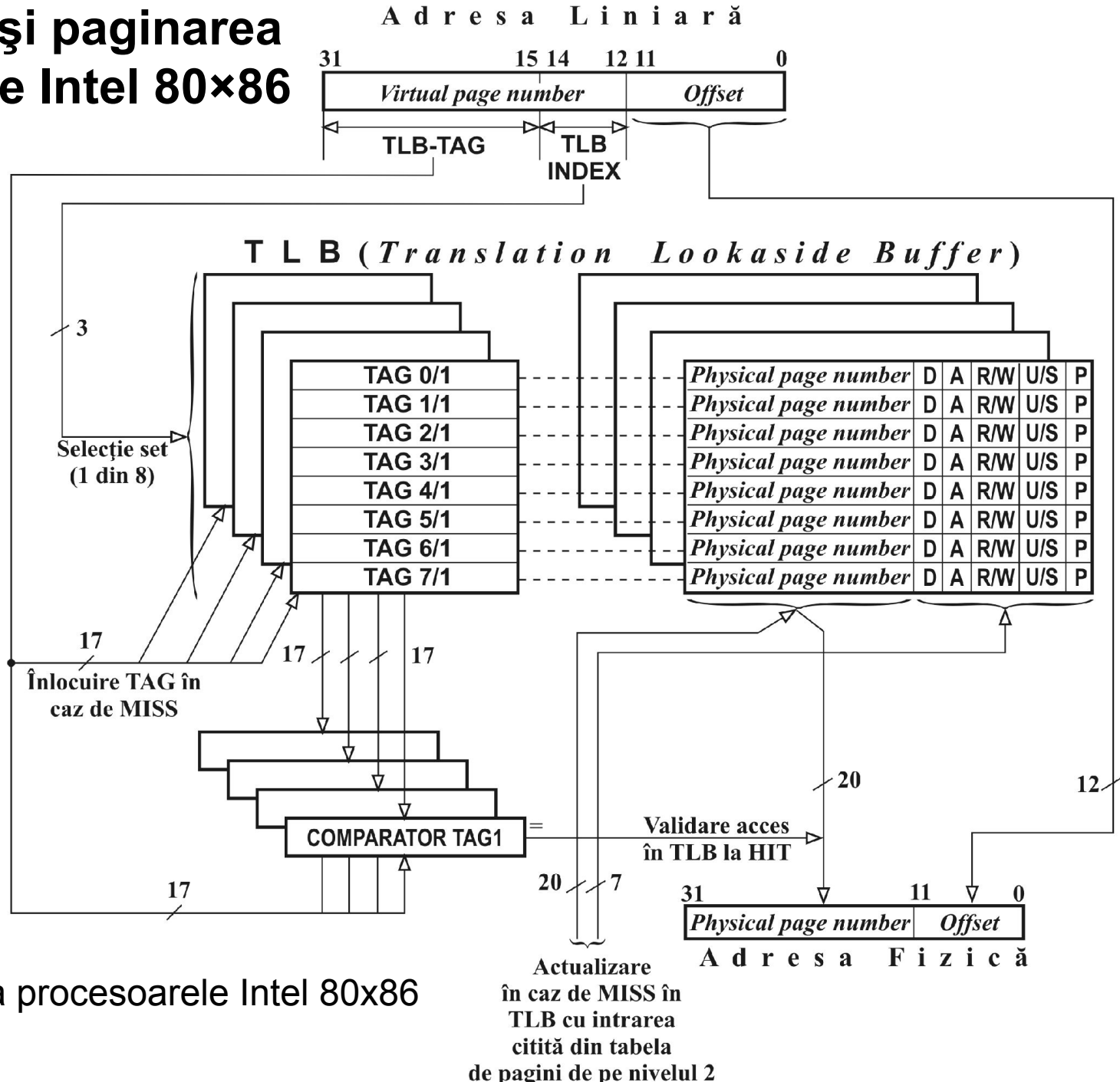
Intrarea în pagina de directori		Intrarea în tabela de pagini de pe nivelul 2		Efectul combinat	
Privilegiu	Tip acces	Privilegiu	Tip acces	Privilegiu	Tip acces
U	R-O	U	R-O	U	R-O
U	R-O	U	R/W	U	R-O
U	R/W	U	R-O	U	R-O
U	R/W	U	R/W	U	R/W
U	R-O	S	R-O	U	R-O
U	R-O	S	R/W	U	R-O
U	R/W	S	R-O	U	R-O
U	R/W	S	R/W	U	R/W
S	R-O	U	R-O	U	R-O
S	R-O	U	R/W	U	R-O
S	R/W	U	R-O	U	R-O
S	R/W	U	R/W	U	R/W
S	R-O	S	R-O	S	R/W
S	R-O	S	R/W	S	R/W
S	R/W	S	R-O	S	R/W
S	R/W	S	R/W	S	R/W

Combinarea protecțiilor definite la nivelul paginii de directori cu cele definite la nivelul tabelii de pagini de pe nivelul 2 la procesoarele Intel 80x86

La Pentium dacă bitul WP (*write protect*) din registrul de control CR0 este setat

Segmentarea și paginarea la procesoarele Intel 80x86

5. TLB

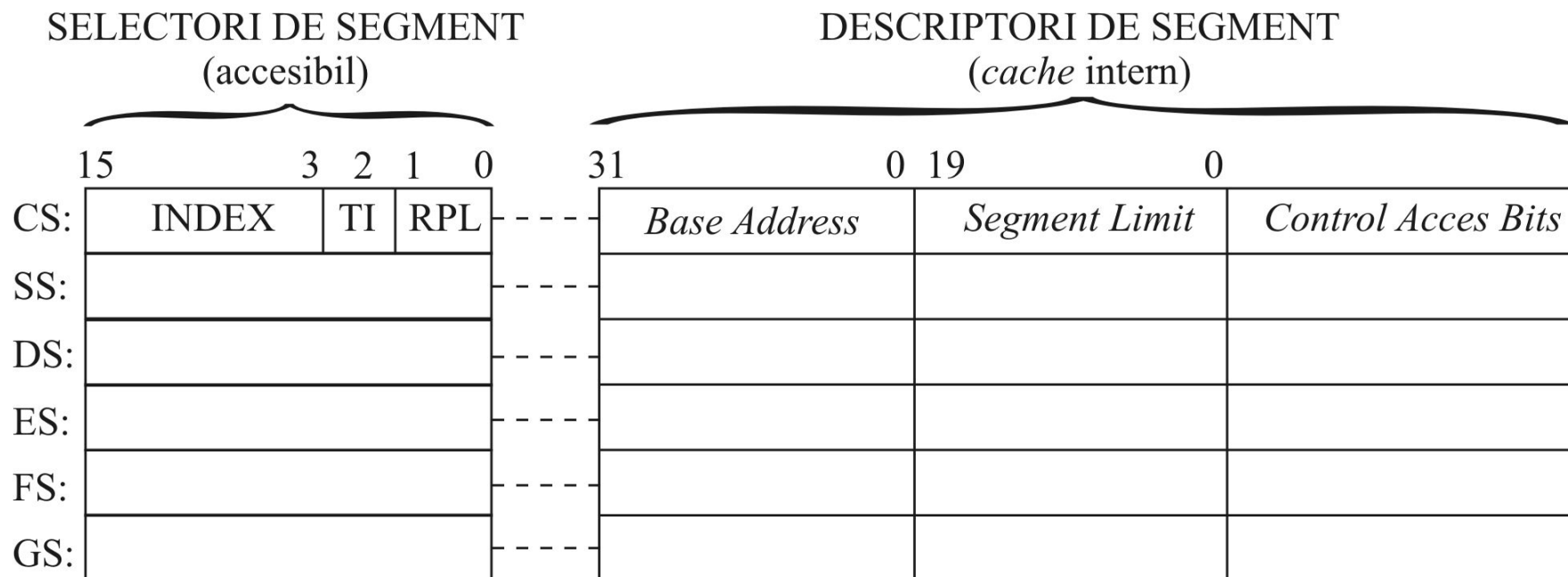


Organizarea TLB la procesoarele Intel 80x86

Segmentarea și paginarea la procesoarele Intel 80x86

6. Registrele de segment

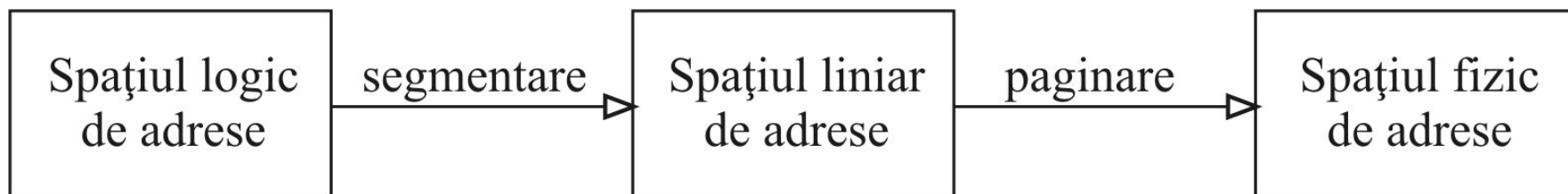
În procesul de traducere a adresei, descriptorul de segment (adresa de bază a segmentului) se obține din partea “ascunsă” (partea *cache*) a registrelor de segment. Pentru a putea accesa locațiile unui segment, selectorul segmentului respectiv trebuie încărcat (instrucțiune *load*) în partea vizibilă a registrului de segment. Faza de execuție a unei asemenea instrucțiuni *load* continuă cu încărcarea descriptorului respectivului segment (preluat din GDT sau LDT) în partea *cache* a registrului.



Structura registrelor de segment la microprocesoarele Intel 80x86

Segmentarea și paginarea la procesoarele Intel 80×86

7. Translatarea spațiului logic de adrese aferent unui *task* în spațiu fizic



Pentru ca două sau mai multe *task*-uri să partajeze date, acestea trebuie să utilizeze un mecanism de segmentare partajat, urmat de un mecanism de paginare partajat.

Există 3 căi posibile de a crea un mecanism partajat de translatare a spațiului logic în spațiu liniar:

- prin descriptorii de segment din GDT
- prin tabele locale (LDT) partajate
- prin descriptori de segment identici plasați în LDT-uri diferite

Pentru ca *task*-urile să partajeze cu adevărat date, mecanismul de segmentare partajat trebuie urmat de un mecanism de paginare partajat. Mecanismul de paginare partajat se poate implementa prin partajarea unor tabele de pagini.

Segmentarea și paginarea la procesoarele Intel 80×86

7. Translatarea spațiului logic de adrese aferent unui *task* în spațiu fizic

